

Introduction to Rust

CSE552 Program Analysis — Lecture 2

Jaemin Hong

What is Rust?

- ▶ A systems programming language to replace C and C++
- ▶ Performance comparable to C and C++
- ▶ Memory safety guaranteed by type checking at compile time
- ▶ Language features adopted from functional languages: closures, algebraic data types (enums), generics, traits



Rust in the Real World

- ▶ The White House recommended using safe languages, including Rust, instead of C and C++¹
- ▶ Many new systems, including operating systems and web browsers, have been developed in Rust
- ▶ Linux has officially added support for Rust in kernel development
- ▶ Coreutils have been rewritten in Rust and installed by default in some recent operating systems, such as Ubuntu 25.10

¹Back to the building blocks: A path toward secure and measurable software (White House, 2024)

Rust in This Course

- ▶ Rust is both an implementation language and a target language for static analysis in this course
 - ▶ In assignments, you will implement a static analyzer for Rust programs in Rust
 - ▶ For the term project, you can use any language and analyze any language
- ▶ Functional languages (e.g., Scala and OCaml) have typically been used for implementing static analyzers
- ▶ Rust provides useful features for implementing static analyzers, while also allowing performant implementations

Variables

- ▶ `let [id] = [expr];` — immutable variable
- ▶ `let mut [id] = [expr];` — mutable variable

Immutable:

```
fn main() {  
    let x = 1;  
    x = 2; // error  
    println!("{x}");  
}
```

Mutable:

```
fn main() {  
    let mut x = 1;  
    x = 2; // ok  
    println!("{x}");  
}
```

Functions

- ▶ `fn [id]([id]: [type], ...) -> [type] { ... }`
- ▶ All parameter types and the return type must be specified
- ▶ The last expression in the function body is the return value

```
fn add(x: i32, y: i32) -> i32 {  
  x + y  
}
```

Types

- ▶ Signed integers: `i8`, `i16`, `i32`, `i64`, `i128`, `isize`
- ▶ Unsigned integers: `u8`, `u16`, `u32`, `u64`, `u128`, `usize`
- ▶ Floating-point numbers: `f32`, `f64`
- ▶ Boolean: `bool`
- ▶ Unit: `()`
- ▶ Character: `char`
- ▶ String: `String`
- ▶ Tuples: `(T1, T2, ...)`
- ▶ Arrays: `[T; N]`
- ▶ Vectors: `Vec<T>`
- ▶ Hash maps: `HashMap<K, V>`

Expression-oriented Programming

- Every expression evaluates to a value

```
let x = {  
  let y = 10;  
  y * y  
};
```

```
let x = if y >= 0 {  
  println!("y is non-negative");  
  y  
} else {  
  println!("y is negative");  
  -y  
};
```


Loops

- ▶ `loop { ... }`
- ▶ `while [expr] { ... }`
- ▶ `for [id] in [expr] { ... }`

```
let mut x = 0;
let y = loop {
  x += 1;
  if x == 10 {
    break x * 2;
  }
};
```

```
let v = vec![1, 2, 3];
for x in v {
  println!("{x}");
}
```

Ownership

- ▶ Each value is owned by at most one variable
- ▶ When the value is assigned to another variable or passed to a function, the ownership is transferred (moved)
- ▶ The compiler prevents using a variable after its ownership has been moved

```
let s = "hi".to_string();  
println!("{s}"); // ok  
let t = s;  
println!("{s}"); // error  
println!("{t}"); // ok
```

Copy and Clone

- ▶ Some types (e.g., primitive types) implement Copy, which allows copying instead of moving

```
let x = 1;
println!("{x}"); // ok
let y = x;
println!("{x}"); // ok
println!("{y}"); // ok
```

- ▶ For types not implementing Copy (typically heap-allocated data), use clone to create a deep copy

```
let s = "hi".to_string();
println!("{s}"); // ok
let t = s.clone();
println!("{s}"); // ok
println!("{t}"); // ok
```

Ownership and Use-After-Free

- The notion of ownership prevents use-after-free

```
let s = "hi".to_string();  
let t = s;  
drop(t); // deallocate the heap memory  
println!("{s}"); // error
```

Borrowing

- ▶ Borrowing allows temporarily using a value without taking ownership
- ▶ `&[expr]` creates a shared reference (read-only) of type `&T`
- ▶ `&mut [expr]` creates a mutable reference (read-write) of type `&mut T`

```
fn show(s: &String) {  
    println!("{s}");  
}  
let s = "hi".to_string();  
show(&s);  
println!("{s}"); // ok
```

```
fn push_dot(  
    s: &mut String  
) {  
    s.push('.');  
}  
let mut s =  
    "hi".to_string();  
push_dot(&mut s);  
println!("{s}"); // ok
```

Aliasing XOR Mutability

- ▶ Multiple shared references to the same value can coexist
- ▶ A mutable reference to a value cannot coexist with any other reference (shared or mutable) to the same value
- ▶ This discipline also prevents use-after-free

```
let mut v = vec![1, 2, 3];  
let r = &v[0];  
v.push(4); // may reallocate the vector  
println!("{r}"); // error
```

Interior Mutability

- ▶ Some types allow mutating the value even through a shared reference
- ▶ `Cell<T>`: provides `get` and `set` methods
- ▶ `RefCell<T>`: provides `borrow` and `borrow_mut` methods that enforce the aliasing XOR mutability rule at runtime

```
let x = Cell::new(1);  
let r1 = &x;  
let r2 = &x;  
r1.set(2);  
println!("{}", r2.get());
```

- ▶ Useful for certain purposes, e.g., caching and cyclic structures

Function Pointers

- ▶ Type: `fn([type], ...) -> [type]`
- ▶ Can be called or passed to higher-order functions

```
fn is_neg(x: &i32) -> bool {  
    *x < 0  
}  
  
let f: fn(&i32) -> bool = is_neg;  
println!("{}", f(&-1));  
  
let mut v = vec![1, -2, 3];  
v.retain(f); // equivalent to 'v.retain(is_neg);'
```


Anonymous Functions and Closures

Anonymous Functions:

- ▶ Functions without a name
- ▶ `|[id], ...| [expr]`

```
let mut v = vec![1, -2];  
v.retain(|x| *x < 0);
```

Closures:

- ▶ Anonymous functions can capture variables from the surrounding scope

```
let n = 10;  
let mut v = vec![5, 15];  
v.retain(|x| *x > n);
```

Structs

- ▶ Custom data types with named fields

```
struct Point { x: f64, y: f64 }  
let p = Point { x: 1.0, y: 2.0 };  
println!("{}", p.x, p.y);
```

- ▶ Structs can have methods

```
impl Point {  
    fn distance_from_origin(&self) -> f64 {  
        (self.x * self.x + self.y * self.y).sqrt()  
    }  
}  
let p = Point { x: 3.0, y: 4.0 };  
println!("{}", p.distance_from_origin());
```

Enums

- ▶ Custom data types that can be one of several variants
- ▶ Used with pattern matching

```
enum Color { Red, Green, Blue }  
let c = Color::Green;  
let name = match c {  
    Color::Red => "red",  
    Color::Green => "green",  
    Color::Blue => "blue",  
};
```

Enums (cont.)

- Each variant can have associated data

```
enum Shape {  
    Circle(f64),  
    Rectangle(f64, f64),  
}  
  
let s = Shape::Circle(1.0);  
let area = match s {  
    Shape::Circle(r) => PI * r * r,  
    Shape::Rectangle(w, h) => w * h,  
};
```

- if let is useful when we only care about one variant

```
if let Shape::Circle(r) = s {  
    println!("circle with radius {r}");  
}
```

Options

- ▶ `Option<T>` is an enum that can be either `Some(T)` or `None`
- ▶ Represents optional existence of a value or the possibility of failure

```
let map = HashMap::from([("a", 1), ("b", 2)]);
match map.get("a") {
    Some(x) => println!("value: {x}"),
    None => println!("key not found"),
}
```

Results

- ▶ Similar to `Option<T>`, but provides information about errors
- ▶ `Result<T, E>` can be either `Ok(T)` or `Err(E)`

```
fn divide(x: i32, y: i32) -> Result<i32, String> {  
    if y == 0 {  
        Err("division by zero".to_string())  
    } else {  
        Ok(x / y)  
    }  
}
```

Error Handling

- ▶ Rust does not have exceptions; errors should be explicitly expressed using `Option` or `Result`
 - ▶ `panic`: unrecoverable error, terminates the program (not intended to be handled)
- ▶ Error propagation can be done using the `?` operator

```
fn add_values_associated_with_keys(
    key1: i32,
    key2: i32
    map: &HashMap<i32, i32>,
) -> Option<i32> {
    let v1 = map.get(&key1)?;
    let v2 = map.get(&key2)?;
    Some(v1 + v2)
}
```

Generics

- Functions can be parameterized over types

```
fn pop_until<T>(v: &mut Vec<T>, n: usize) {  
    while v.len() > n {  
        v.pop();  
    }  
}
```

- User-defined types can also be generic

```
struct Point<T> { x: T, y: T }  
let p1: Point<f64> = Point { x: 1.0, y: 2.0 };  
let p2: Point<i32> = Point { x: 1, y: 2 };
```


Traits

- ▶ A trait represents a set of types that share common behavior
 - ▶ Resembles type classes in Haskell
 - ▶ Similar to interfaces or abstract classes in object-oriented languages

```
trait HasArea {  
    fn area(&self) -> f64;  
}  
  
struct Circle { radius: f64 }  
impl HasArea for Circle {  
    fn area(&self) -> f64 { PI * self.radius * self.radius }  
}  
  
struct Rectangle { width: f64, height: f64 }  
impl HasArea for Rectangle {  
    fn area(&self) -> f64 { self.width * self.height }  
}  
  
fn larger<T: HasArea>(a: T, b: T) -> T {  
    if a.area() > b.area() { a } else { b }  
}
```

Lifetimes

- ▶ A lifetime represents how long a reference is valid
- ▶ `&'a T` or `&'a mut T` indicates that the reference is valid for the lifetime `'a`
- ▶ Lifetime annotations can be omitted in many cases, but sometimes they are required to disambiguate lifetimes

```
fn foo<'a, 'b>(x: &'a i32, y: &'b i32) -> &'a i32 {x}
```

- ▶ Types are parameterized over lifetimes when they contain references

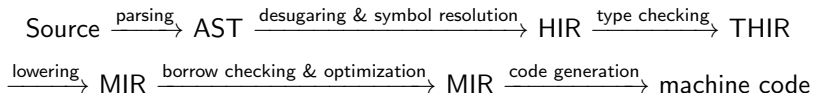
```
struct Point<'a> { x: &'a f64, y: &'a f64 }
```

References

- ▶ The Rust Programming Language:
<https://doc.rust-lang.org/book>
- ▶ Rust By Example:
<https://doc.rust-lang.org/rust-by-example/>
- ▶ The Rust Standard Library:
<https://doc.rust-lang.org/std>
- ▶ The Rust Reference:
<https://doc.rust-lang.org/reference>
- ▶ Rust Playground: <https://play.rust-lang.org/>

The Rust Compiler (rustc)

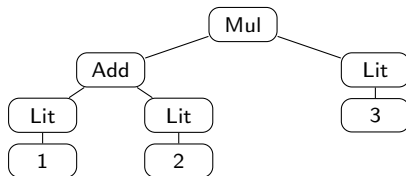
- ▶ The compiler consists of several passes that transform the source code into machine code:



- ▶ It exposes its internal data structures and passes as APIs, which can be used for implementing static analyzers
- ▶ Choosing the right code representation is important for implementing a static analyzer

Abstract Syntax Tree (AST)

- ▶ Represents code as a tree, not text
- ▶ Similar to source code, but:
 - ▶ Macros are expanded
 - ▶ Submodules implemented in other files are loaded (a single AST for the whole crate)
- ▶ Example: $(1 + 2) * 3$



(Conceptual example, not the actual AST used in the compiler)

- ▶ You can see the AST using `rustc -Z unpretty=ast-tree,expanded [file.rs]`

High-level Intermediate Representation (HIR)

- ▶ Similar to AST, but:
 - ▶ Desugared (e.g., `for` becomes `loop`)
 - ▶ Symbols are resolved

```
let x = 1;
{
  let x = 2;
  println!("{x}");
}
```

- ▶ What does this `x` refer to?
- ▶ You can see the HIR using `-Z unpretty=hir (text)` or `-Z unpretty=hir-tree (tree)`

Typed High-level Intermediate Representation (THIR)

- ▶ Similar to HIR, but:
 - ▶ Overloading is resolved
 - ▶ Implicit coercions are made explicit

HIR:

```
fn foo(  
  x: &mut i32  
) -> &i32 {  
  x  
}
```

THIR:

```
fn foo(  
  x: &mut i32  
) -> &i32 {  
  &*x  
}
```

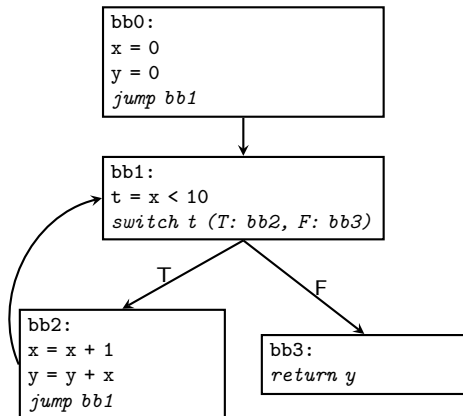
- ▶ You can see the THIR using `-Z unpretty=thir-tree`

Mid-level Intermediate Representation (MIR)

- ▶ A control-flow graph (CFG) representation
 - ▶ A function consists of basic blocks and edges between them
 - ▶ Each basic block consists of a sequence of statements and a terminator
 - ▶ Statement: assignment
 - ▶ Terminator: jump, switch, call, return

Mid-level Intermediate Representation (MIR) — Example

```
let mut x = 0;  
let mut y = 0;  
while x < 10 {  
    x += 1;  
    y += x;  
}  
return y;
```



(Conceptual example, not the actual MIR used in the compiler)

- You can see the MIR using `-Z unpretty=mir`

Comparison of Code Representations

- ▶ AST, HIR, THIR vs. MIR:
 - ▶ MIR is suitable for most analyses because its CFG structure makes execution order explicit and it has fewer language constructs
 - ▶ If the execution order does not matter (flow-insensitive analysis), other representations may be used, especially when results should be close to the source code
- ▶ AST vs. HIR, THIR:
 - ▶ Symbols are not resolved in AST, so HIR and THIR are more convenient
- ▶ HIR vs. THIR:
 - ▶ HIR is useful when traversing parent nodes is required or even ill-typed code should be analyzed
 - ▶ THIR is useful if type information is important
- ▶ In this course, MIR is used in most cases, but HIR is used for type analysis

Important Types in HIR

- ▶ `LocalDefId`: a unique identifier for a top-level item in the crate
- ▶ `DefId`: a unique identifier for a top-level item in any crate, including dependencies. ($\text{DefId} = \text{LocalDefId} + \text{crate ID}$)
- ▶ `HirId`: a unique identifier for any node in the HIR; each local variable has a unique `HirId` as well
- ▶ `Item`: a top-level item
- ▶ `Stmt`: a statement
- ▶ `Expr`: an expression
- ▶ `Visitor`: a trait for traversing the HIR; you can implement your own logic by defining a struct that implements this trait

Important Types in MIR

- ▶ **Body**: a function in MIR
- ▶ **BasicBlock**: a unique identifier for a basic block
- ▶ **BasicBlockData**: a basic block
- ▶ **Statement**: a statement
- ▶ **Terminator**: a terminator
- ▶ **Place**: a memory location; can be used as the LHS of an assignment
- ▶ **Rvalue**: the RHS of an assignment
- ▶ **Operand**: an operand used in an Rvalue (a place or a constant)
- ▶ **Const**: a constant value

Important Types in MIR (cont.)

- ▶ **Local**: a unique identifier for a local variable
 - ▶ Place is a Local with projections (e.g., field access)
 - ▶ Each Local is represented as an integer
 - ▶ `_0` is the return value; `_1`, `_2`, ... are parameters

```
fn add(x: i32, y: i32) -> i32 {  
    x + y  
}
```

```
_0 = Add(_1, _2);  
return;
```

- ▶ **Visitor**: a trait for traversing the MIR; you can implement your own logic by defining a struct that implements this trait

TyCtxt

- ▶ The central data structure of the compiler²
- ▶ When invoking the compiler, a TyCtxt value is given, and many APIs are provided as methods of TyCtxt
- ▶ Examples:
 - ▶ `hir_visit_all_item_likes_in_crate`: visits all item-likes in the crate in some deterministic order
 - ▶ `parent_hir_id`: returns the `HirId` of the parent HIR node of node with the given `hir_id`
 - ▶ `optimized_mir`: MIR after the optimization passes have run

²https://doc.rust-lang.org/stable/nightly-rustc/rustc_middle/ty/context/struct.TyCtxt.html

References

- ▶ HIR: https://doc.rust-lang.org/stable/nightly-rustc/rustc_hir/hir
- ▶ MIR: https://doc.rust-lang.org/stable/nightly-rustc/rustc_middle/mir
- ▶ Rust Compiler Development Guide:
<https://rustc-dev-guide.rust-lang.org/>

Summary

- ▶ Rust is a systems programming language with memory safety guaranteed by the type system at compile time
- ▶ Key language features: variables, functions, types, ownership, borrowing, enums, traits, lifetimes
- ▶ The rustc compiler pipeline: $AST \rightarrow HIR \rightarrow THIR \rightarrow MIR \rightarrow \text{machine code}$
- ▶ MIR (control-flow graph) is most suitable for static analysis due to explicit execution order and fewer language constructs
- ▶ Key types for analysis:
 - ▶ HIR: `HirId`, `Item`, `Expr`
 - ▶ MIR: `Body`, `BasicBlockData`, `Statement`, `Terminator`, `Place`, `Rvalue`, `Operand`, `Const`, `Local`
 - ▶ `LocalDefId`, `DefId`, `TyCtxt`, `Visitor`